

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Sistemas Operativos

Ano Letivo de 2025/2026

Relatório - SO

Afonso Barros
A112178

Enrico Prazeres
A112881

Gonçalo Fernandes
A111855

May 10, 2026

LI3

Índice

1. Introdução **1**

2. Desenvolvimento **2**

2.1. Arquitetura Geral 2

2.1.1. Runner 3

2.1.2. Controller 3

2.2. Mecanismos de Comunicação (FIFOs) 4

2.2.1. Estrutura de Pedidos 4

2.2.2. Pipe Principal do Controller 4

2.2.2.1. Gestão de Ciclo de Vida e Sincronização 4

2.2.3. FIFOs Privados dos Runners 5

2.2.4. Fluxo de Comunicação 5

2.3. Motor de Execução e Parsing 6

2.3.1. Parsing do Comando 6

2.3.2. Encadeamento com Pipes Anónimos 6

2.3.3. Redirecionamentos com dup2 6

2.3.4. Isolamento do Controller 6

2.4. Algoritmos de Escalonamento 6

2.4.1. FCFS - First Come, First Served 7

2.4.2. RR - Round Robin 7

2.4.3. LCF - Least Commands First 7

2.4.4. LET - Less Execution Time 8

2.4.5. Histórico Persistente 8

2.5. Testes 8

2.5.1. Metodologia de Teste 8

2.5.1.1. Configuração dos Testes 8

2.5.1.2. Carga de Trabalho 9

2.5.1.3. Histórico **`Cold`** e **`Warm`** 9

2.5.1.4. Métricas de Avaliação 9

2.5.2. Análise de Resultados 10

2.5.2.1. Impacto do Paralelismo 10

2.5.2.2. Comparação de Políticas a Baixo Paralelismo 11

2.5.2.3. Penalização Relativa (*Slowdown*) 12

2.5.2.4. Justiça entre Utilizadores 13

2.5.2.5. Métricas Agregadas de Justiça 14

2.5.2.6. Impacto do Histórico Persistente 15

2.5.3. Conclusão da Análise 16

3. Conclusão **17**

1. Introdução

O presente relatório documenta o trabalho prático da unidade curricular **Sistemas Operativos**, lecionada no 2.º ano da Licenciatura em Engenharia Informática da Universidade do Minho, durante o ano letivo 2025/2026.

O projeto consiste no desenvolvimento de um ambiente de execução de comandos *bash* gerido por um controlador centralizado. O sistema é composto por dois programas distintos: o *runner* e o *controller*, que comunicam exclusivamente através de *Named Pipes* (FIFOs).

Ao longo deste relatório, será apresentada a arquitetura de processos adotada, os mecanismos de comunicação implementados e os algoritmos de escalonamento desenvolvidos, culminando numa análise crítica das decisões tomadas e das suas implicações práticas.

2. Desenvolvimento

O sistema desenvolvido implementa uma solução completa para a orquestração de execução de comandos, estruturada em dois componentes principais: o *runner* e o *controller*, e um conjunto de módulos internos que encapsulam as responsabilidades de escalonamento, comunicação e execução.

O *runner* atua como cliente: permite criar um pedido para a execução de um comando por parte do utilizador, notifica o *controller* da sua intenção de o executar e aguarda autorização antes de prosseguir para a execução. O *controller*, por sua vez, gere de forma concorrente os pedidos provenientes de múltiplos *runners*, aplicando uma política de escalonamento configurável, controlando o grau de paralelismo e mantendo um registo persistente de *logs* da atividade do sistema.

2.1. Arquitetura Geral

O sistema organiza-se em torno de dois programas independentes que interagem através de *Named Pipes*. A Figura 1 ilustra a arquitetura geral do sistema, evidenciando o fluxo de comunicação entre utilizadores, *runners* e o *controller*.

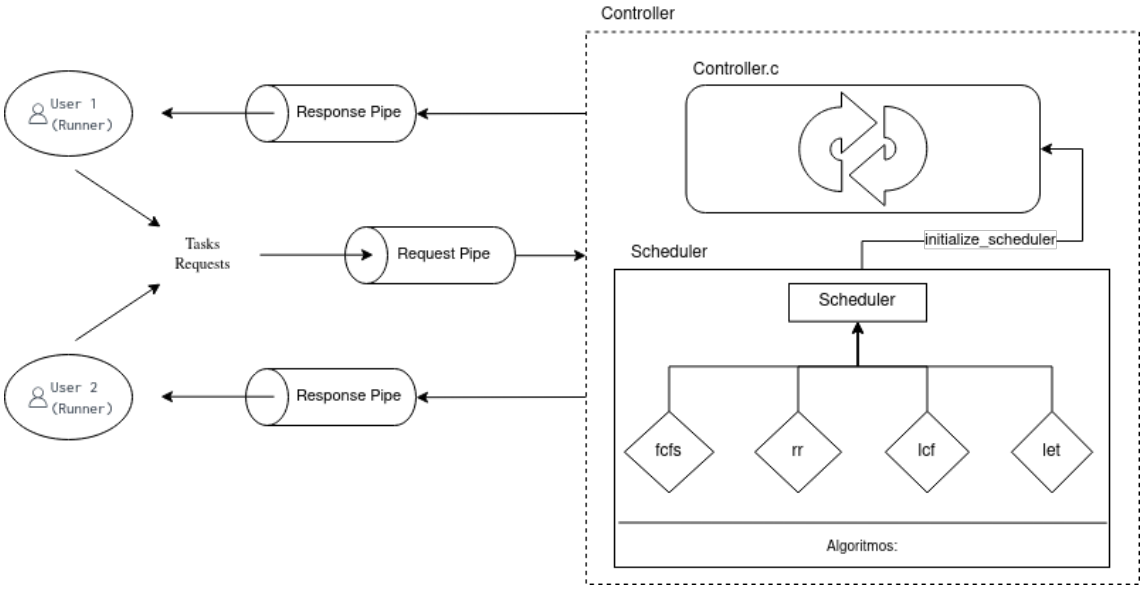


Figura 1: Diagrama da Arquitetura do Projeto

Envio de Pedidos: Do lado esquerdo, observamos que todos os runners (User 1, User 2, etc.) partilham um único canal de entrada no sistema, o "Request Pipe". É através desta pipe partilhada que todos os utilizadores enviam simultaneamente os seus pedidos (estruturas Request) para o controller.

Receção de Respostas: Para garantir que as respostas não se misturam e que cada runner recebe apenas a informação que lhe pertence, o sistema utiliza uma abordagem dedicada no regresso. O diagrama ilustra múltiplos "Response Pipes". Na prática, isto significa que é criada uma named pipe individual para cada runner, identificada de forma única pelo seu PID (número de processo). É através deste canal privado que o controller envia as mensagens e autorizações diretamente para o respetivo runner.

Processamento Interno (Abstração do Scheduler): Do lado direito, dentro do bloco do controller, encontra-se a lógica de escalonamento e despacho. A arquitetura foi desenhada com um elevado nível de abstração. No arranque do sistema, o controller inicializa a estrutura central do Scheduler, passando-lhe o número do algoritmo escolhido. Neste momento, o sistema carrega dinamicamente os ponteiros de função correspondentes a apenas um dos quatro algoritmos escolhidos (FCFS, RR, LCF ou LET). Após

esta configuração inicial, o controller já não precisa de conhecer os detalhes da política ativa; ele interage com o Scheduler através de uma interface genérica unificada. À medida que o controlador pode ter mais comandos a executar, ele pede ao Scheduler o próximo comando, independentemente da lógica que esta por trás.

2.1.1. Runner

O *runner* é um processo curto, lançado pelo utilizador para submeter um único pedido e termina após a sua execução. Suporta três modos de operação, selecionados através de uma *flag* passada como argumento:

- **`-e <user-id> "<comando> [args...]“`**: Submete um comando para execução. O *runner* cria o seu FIFO de resposta privado, envia uma instância da estrutura `Request` para o pipe principal do *controller* e bloqueia à espera de autorização. Após receber o sinal, executa o comando e, no fim, notifica o *controller* com a duração medida.
- **`-c`**: Consulta o estado atual do sistema, listando os comandos em execução e em espera. A lista dos comandas por executar é feita de forma ordenada de acordo com o seria a sua ordem esperada naquele momento. O controlador cria um processo filho responsável por tratar de listar e enviar a resposta para o respetivo *runner* (mais detalhes na secção 2.1.2).
- **`-s`**: Solicita o encerramento do controller. Ao receber este pedido, o controller define internamente uma flag de encerramento e envia a si próprio uma mensagem de FLUSH através do pipe principal. Devido à natureza sequencial (FIFO) do pipe, esta mensagem de FLUSH é inserida no final da fila de mensagens. Isto garante que quaisquer pedidos que já se encontrassem em trânsito no pipe (como notificações WAIT de processos filho ou comandos submetidos na fração de segundo antes do unlink do FIFO) sejam obrigatoriamente lidos e processados antes da mensagem de FLUSH. Ao ler finalmente o FLUSH, o controller aciona o fecho do seu descritor de escrita artificial (ver secção 2.2.2.1 onde isto é explicado em detalhe). O controller só termina após todos os runners em execução concluírem os seus comandos, o scheduler esvazia a fila de comandos pendentes e todos as *requests* que tenham chegado antes do *flush* terem sido executadas, garantindo que nenhum comando previamente admitido é descartado. Após o encerramento completo, o controller notifica o runner solicitante com a mensagem ENDED no seu FIFO privado.

A identificação única de cada *runner* é o seu *PID* de sistema, que serve não só como identificador de comando, mas também como sufixo do FIFO de resposta privado (`tmp/runner_<PID>`).

2.1.2. Controller

O *controller* é um processo de longa duração, executado com dois argumentos: o número máximo de comandos a executar em paralelo e o identificador da política de escalonamento. A lógica central consiste num ciclo principal que, a cada iteração, decide se deve bloquear à espera de um novo pedido no pipe principal ou avançar para a execução de comandos pendentes.

A condição de bloqueio é avaliada pela função `check_wait_condition`, onde o *controller* bloqueia se o limite de paralelismo já tiver sido atingido, se não houver comandos pendentes na fila, ou se ainda existirem *runners* em execução após ter recebido o sinal de encerramento. Caso contrário, avança para `dispatch_pending_commands`, que extrai o próximo comando do *scheduler* e notifica o *runner* correspondente.

O processamento de pedidos de consulta (`-c`) é entregue a um processo filho criado através de um `fork`. Uma vez que o processo filho herda uma cópia do espaço de endereçamento do pai no momento do `fork`, deixa de ser necessário criar *snapshots* explícitos. Assim, o processo filho pode listar ambos os comandos em execução e pendentes diretamente sobre essas cópias, sem que haja o risco de alterar as estruturas originais do *controller*. Posto isto, esta propriedade é aproveitada deliberadamente durante a implementação da listagem nos algoritmos de escalonamento, não necessitando de qualquer mecanismo de bloqueio ou cópia das estruturas internas.

O *schedule* mantém ainda dois registos auxiliares: uma `GQueue` com os comandos atualmente em execução (indexada por uma `GHashTable` para remoção em *O(1)*) e um ficheiro de *log* persistente onde cada comando terminado é registado com o seu identificador, utilizador, duração de execução e tempo total desde a submissão.

2.2. Mecanismos de Comunicação (FIFOs)

Toda a comunicação entre *runners* e *controller* assenta exclusivamente em *Named Pipes* (*FIFOs*), criados com `mkfifo` e acedidos com as chamadas de sistema `open`, `read` e `write`.

2.2.1. Estrutura de Pedidos

O protocolo de comunicação é definido pela estrutura `Request`, com um tamanho fixo:

```
struct request {
  Status status;         // EXECUTE | TERMINATED | LIST | END |FLUSH | WAIT
  int    user_id;
  int    command_id;     // PID do runner
  long   created_timestamps;
  long   duration;
};
```

O campo `status` determina o tipo de pedido, já os restantes campos transportam os dados necessários para o escalonamento e registo do pedido. Ademais, a transmissão de uma estrutura de tamanho fixo *(36 bytes)* simplifica significativamente a lógica de leitura, eliminando a necessidade de delimitadores ou *parsing*. Adicionalmente, como o *POSIX* garante escrita atómica para mensagens até *PIPE_BUF* (4096 bytes no Linux), e cada *Request* ocupa apenas 36 bytes, todas as escritas no *pipe* são atómicas por definição, não havendo risco de entrelaçamento entre mensagens de diferentes *runners* concorrentes.

2.2.2. Pipe Principal do Controller

O *controller* mantém um único FIFO, `tmp/controller_pipe`. Este pipe é o ponto de entrada de todos os pedidos do sistema.

O pipe é aberto pelo *controller* em modo de leitura (`fd_read`) e, adicionalmente, em modo de escrita (`fd_write`) pelo próprio *controller*. Este segundo descritor, designado artificial, é mantido aberto durante todo o ciclo de vida do *controller*.

Este mecanismo evita o problema clássico dos *Named Pipes*, onde por norma, quando o último *writer* fecha o seu descritor, todos os *readers* recebem imediatamente *EOF* (`read()` devolve 0). Sem o descritor artificial, bastaria um momento em que nenhum *runner* tivesse o *pipe* aberto para o *controller* receber *EOF* e entrar num *loop* incontrolável de leituras vazias. Com o *fd_write* sempre aberto no *controller*, o *pipe* nunca fica sem *writers* enquanto o *controller* estiver ativo.

2.2.2.1. Gestão de Ciclo de Vida e Sincronização

Durante o desenvolvimento do sistema, a gestão de processos e a sincronização da terminação revelaram-se desafios críticos que exigiram soluções de engenharia específicas para garantir a robustez do controller.

Prevenção de Esgotamento de Processos: Inicialmente, a arquitetura previa a recolha de processos filho (waitpid) apenas no final da execução do ciclo principal do controller. No entanto, sendo este um programa com características de servidor, a acumulação de processos terminados mas não recolhidos (processos 'zombie') resultantes dos comandos de consulta (-c) poderia levar rapidamente ao esgotamento do limite máximo de processos permitidos pelo sistema.

Para resolver esta vulnerabilidade, foi implementado o seguinte mecanismo: antes de terminar, cada processo filho responsável por uma listagem envia uma estrutura Request com o estado WAIT e o seu

próprio PID para o pipe principal. O processo pai, ao ler este pedido no seu ciclo normal de operação, executa o waitpid de forma dirigida. Como esta notificação é enviada no último instante da vida do filho, o bloqueio do pai é nulo, sendo no pior dos casos um instante ínfimo, permitindo a libertação imediata de recursos sem comprometer a disponibilidade do servidor.

Protocolo de Encerramento e Sincronização Final: A introdução do mecanismo de WAIT assíncrono trouxe uma complexidade adicional à fase de encerramento. Após a receção do comando END (emitido por um runner que foi rodado com a flag -s), o controller deve assegurar que o seu descritor de escrita (fd_write) permanece aberto enquanto houver processos de listagem ativos, de forma a garantir que estes consigam ser devidamente terminados.

A solução implementada para garantir uma terminação limpa baseia-se em três partes:

1) Unlink: No momento em que o pedido END é processado, o controller remove o FIFO do sistema de ficheiros (unlink). Isto impede a criação de novas ligações por parte de outros runners, mas mantém os descritores já abertos em funcionamento.

2) Persistência da Comunicação nos Runners: Esta alteração exigiu que os *runners* mantivessem o seu descritor para a pipe do controller aberta entre ambas as comunicações com este. Por exemplo se o *runner* fechasse o descritor depois de receber a autorização para executar podia não conseguir voltar a abri-lo para informar ao *controller* que já terminou, se *entretanto* ele tivesse recebido uma *request* com a *flag* -s para acabar (e consequentemente desse *unlink*).

3) Mecanismo de Flush: Para determinar o momento exato em que o descritor de escrita dummy do controller deve ser fechado, introduziu-se o estado FLUSH à request. Quando o controller recebe a flag END, faz o unlink e logo de seguida envia uma mensagem FLUSH para si mesmo através do seu próprio descritor de escrita. Quando o ciclo de leitura do controller processa este FLUSH, ele sabe que pode fechar o seu descritor de escrita com segurança. Como o FIFO já sofreu unlink, nenhum runner novo conseguirá abrir o pipe. Contudo, os runners que já estavam ativos mantêm os seus descritores de escrita abertos. Assim, apenas quando o último desses runners terminar e fechar o seu descritor é que o controller receberá um EOF natural no descritor de leitura, garantindo que todas as requests pendentes foram devidamente tratadas antes da terminação final.

2.2.3. FIFOs Privados dos Runners

Para cada *runner*, é criado um FIFO privado com o caminho `tmp/runner_<PID>`. Este pipe é utilizado pelo *controller* para enviar mensagens unidirecionais ao *runner* específico:

- **`GO`**: Autoriza a execução do comando.
- **`ENDED`**: Confirma ao *runner* de encerramento (`-s`) que o *controller* terminou.
- Listagem de comandos: no caso do pedido `-c`, o *controller* escreve a string completa com o estado do sistema.

Após a leitura da resposta, o *runner* remove o seu FIFO com `unlink` e fecha os descritores necessários garantindo que o diretório `tmp/` não acumula ficheiros de *pipes* antigos.

2.2.4. Fluxo de Comunicação

O ciclo de vida de um comando submetido para execução segue os seguintes passos:

1. O *runner* cria `tmp/runner_<PID>` e envia uma `Request` com `status = EXECUTE` para o *pipe* principal e fica imediatamente à espera da mensagem do *controller*.
2. O *controller* recebe o pedido e adiciona o comando à fila do respetivo *scheduler*.
3. Quando o *scheduler* autoriza a execução, o *controller* envia a notificação `GO` no *FIFO* privado do *runner* e regista o comando como ativo.
4. O *runner* desbloqueia, executa o comando e, após a conclusão, envia uma nova `Request` com `status = TERMINATED` e os valores de duração medidos.

5. O *controller* remove o comando da lista de execução, atualiza o histórico do utilizador, regista a entrada no ficheiro de *log* e decrementa o contador de comandos ativos, libertando consequentemente uma vaga para o próximo comando na fila.

2.3. Motor de Execução e Parsing

O motor de execução é a peça responsável por ler o comando escrito pelo utilizador e transformá-lo em processos reais a correr no sistema, lidando corretamente com pipes (|) e redirecionamentos (>, <, 2>).

2.3.1. Parsing do Comando

Para separar o texto do comando em vários argumentos, usámos a função g_shell_parse_argv da biblioteca GLib (a autorização da utilização desta função foi verificada com o docente da cadeira). Em vez de tentarmos dividir o texto manualmente pelos espaços, esta função faz o parsing exatamente como um terminal bash real, o que nos garante que comandos com aspas (como echo "olá mundo") não são partidos a meio e chegam perfeitos ao vetor de argumentos (argv).

2.3.2. Encadeamento com Pipes Anónimos

Quando o utilizador liga vários comandos (por exemplo, ls | wc), o sistema divide-os no caracter | e cria um pipe anónimo com a chamada pipe() para os ligar. A implementação usa uma lógica sequencial: guardamos sempre o lado de leitura do pipe anterior na variável prev_read_fd. Assim, após o fork(), cada novo processo filho liga o seu stdin a essa leitura anterior, e o seu stdout à escrita do novo pipe. Teve-se em conta o fecho do descritor de escrita por parte do pai nos pipes logo após criá-los, para que os processos filhos recebam a notificação de fim de ficheiro (EOF) e saibam quando devem terminar.

2.3.3. Redirecionamentos com dup2

Para suportar sinais de redirecionamento como >, < e 2>, o processo filho identifica o ficheiro pretendido e abre-o. De seguida, usamos a chamada de sistema dup2() para substituir a entrada ou saída padrão do processo (stdin, stdout ou stderr) por esse ficheiro recém-aberto. Por fim, apagamos esses operadores do vetor de argumentos antes de chamar o execvp(), para que o programa não tente ler os símbolos como se fossem texto normal.

2.3.4. Isolamento do Controller

Como medida crítica de segurança, logo após o fork(), o processo filho fecha o descritor de ficheiro que o ligava ao controller (controller_fd). Isto garante que o programa executado pelo utilizador fica isolado e não tem qualquer forma de enviar lixo ou corromper o pipe principal do sistema.

2.4. Algoritmos de Escalonamento

O *controller* suporta quatro políticas de escalonamento, configuráveis no arranque através do argumento `sched-policy` (0 a 3). Todas partilham a mesma interface funcional, definida no módulo `Scheduler` através de ponteiros para funções (`addCommand`, `getNext`, `has_commands`, `list_commands`, `cleanup`), o que torna o *dispatcher* completamente agnóstico à política em uso, permitindo a integração de novos algoritmos sem que sejam necessárias mudanças nos restantes módulos principais. A implentação dos algortimos teve em conta não só que eles sejam justos mas também que a escolha do próximo comando seja feita de forma eficiente e rápida.

2.4.1. FCFS - First Come, First Served

A política *FCFS* é, dentre as demais, a mais simples. Neste caso os comandos são servidos pela ordem de chegada, sem qualquer consideração sobre o utilizador que os submeteu, ou qualquer outro dado. A implementação utiliza uma `GQueue` como fila FIFO, onde `add_command` insere no fim (`push_tail`) e `get_next` extrai do início (`pop_head`). A complexidade de ambas as operações é *O(1)*.

Esta política é justa no sentido temporal, visto que nenhum comando espera mais do que os que chegaram antes, mas pode favorecer utilizadores que submetam muitos comandos em sequência, penalizando utilizadores ocasionais.

2.4.2. RR - Round Robin

O *Round Robin* implementa uma rotação entre utilizadores ativos, garantindo que nenhum utilizador explora o sistema de forma abusiva quando há vários a submeter comandos simultaneamente. A estrutura de dados central é uma `GSList` de utilizadores ativos combinada com uma `GHashTable` que mapeia cada utilizador para a sua fila de comandos (`GQueue`).

A cada chamada a `get_next`, o *scheduler* serve um comando do utilizador atual e avança o ponteiro para o utilizador seguinte na lista circular. Quando um utilizador esgota os seus comandos pendentes, é removido da lista de utilizadores ativos. Novos utilizadores são inseridos imediatamente antes do utilizador corrente, assegurando que entram na rotação sem passar a vez dos utilizadores já presentes.

Esta política oferece a melhor equidade entre utilizadores distintos, sendo particularmente adequada para ambientes com múltiplos utilizadores a competir pelos mesmos recursos.

2.4.3. LCF - Least Commands First

A política Least Commands First (LCF) dá prioridade aos utilizadores que executaram o menor número de comandos. Para garantir que o sistema é rápido e justo, a nossa implementação atualiza o número de comandos de cada utilizador de forma constante e em tempo real.

Estruturas de Dados

Para gerir a fila de utilizadores de forma eficiente, combinamos duas estruturas da biblioteca GLib:

GHashTable (Acesso imediato em O(1)): Funciona como um dicionário que associa cada user_id aos seus dados. Isto permite-nos encontrar a fila privada de um utilizador instantaneamente, sem ter de procurar um a um.

GSequence (Ordenação automática em O(logn)): Mantém os utilizadores rigorosamente ordenados pelo seu número de comandos. Para saber quem é o próximo utilizador a executar, o controller apenas precisa de retirar o primeiro elemento desta sequência, o que é uma operação imediata (O(1)).

Tivemos o cuidado de fazer o cálculo do número de comandos de forma dinâmica, isto é, considerando os comandos que estão atualmente a serem executados. Isto permite que um utilizador que envie muitos comandos num curto período seja penalizado mesmo na sua ordem de execução mesmo que os seus comandos ainda não tenham terminado de executar. Além disso quando o utilizador submete o primeiro comando, o sistema carrega o seu histórico passado através do ficheiro historical.bin. Permitindo ganhar contexto entre interações e ser justo a longo prazo.

Sempre que um pedido entra, o peso desse utilizador aumenta imediatamente (user->dynamic_command_count++), e a sua posição na GSequence é logo corrigida usando a função g_sequence_sort_changed.

Esta abordagem procura desta forma garantir que as vezes em que processador é utilizado por um utilizador é partilhado de forma justa.

2.4.4. LET - Less Execution Time

O *Less Execution Time* é a política mais complexa onde se prioriza o utilizador com menor tempo de execução acumulado, considerando tanto o histórico persistido como o tempo ativo dos comandos atualmente em execução. Esta distinção é fundamental, já que um utilizador cujo comando já está a correr está a consumir tempo de *CPU*, e esse consumo deve ser contabilizado nas decisões de escalonamento mesmo antes de o comando terminar.

A estrutura de dados central do algoritmo agrega: • `users_commands` (*GHashTable*): fila de comandos pendentes por utilizador. • `active_users` (*GQueue*): lista dos utilizadores com pelo menos um comando pendente. • `active_time_by_user` (*GHashTable*): mapa temporário com o tempo ativo acumulado dos comandos em execução, recalculado a cada decisão de escalonamento. • `users_history`: referência ao módulo histórico persistente. • `scheduler`: referência ao Scheduler para acesso à lista de comandos em execução.

A cada chamada a `get_next`, a função `rebuild_active_time_by_user` percorre a lista de comandos correntemente em execução (`get_running_commands`), calcula para cada um o tempo decorrido desde `started_at` até ao instante atual (`CLOCK_MONOTONIC`) e acumula esse valor por utilizador. O tempo efetivo de um utilizador é então a soma do tempo histórico persistido com o tempo ativo calculado:

effective_time(u) = historical_time(u) + active_time(u)

A função `find_best_user_link_let` varre linearmente a fila de utilizadores ativos e seleciona aquele com o menor `effective_time`, com desempate por `user_id`. Esta abordagem tem complexidade *O(n)* por decisão, onde *n* é o número de utilizadores distintos com comandos pendentes, o que é aceitável dado o carácter potencialmente dinâmico e preciso do cálculo de tempo ativo.

2.4.5. Histórico Persistente

Tal como se verificou acima, os algoritmos LCF e LET recorrem ao módulo `Historical`, que mantém uma `GHashTable` com estatísticas por utilizador (`commands_run` e `total_execution_time`). Este estado é contido num ficheiro binário (`historical.bin`) através de leituras e escritas diretas de estruturas `UserStats` com tamanho fixo. Trata-se de uma abordagem simples e eficiente que não requer qualquer formato de texto. O ficheiro é carregado no arranque com `load_history` e guardado no encerramento com `save_history`, através de `O_TRUNC` para garantir que o ficheiro reflita sempre o estado mais recente.

2.5. Testes

Para validar o comportamento dos algoritmos de escalonamento implementados, e compreender as suas implicações práticas, foi desenvolvida uma metodologia de teste sistemática, composta por dois componentes: um *script* de execução de testes (`run_experiments.sh`) e um *script* de análise estatística (`analyze_stats.py`).

2.5.1. Metodologia de Teste

2.5.1.1. Configuração dos Testes

O *script* `run_experiments.sh` automatiza a execução de um conjunto de testes com todas as combinações relevantes de parâmetros. Para cada configuração, o *controller* é iniciado com uma política de escalonamento e um grau de paralelismo definidos, é submetida uma carga de trabalho, e no final os resultados são registados num ficheiro CSV acumulativo.

As variáveis independentes controladas são:

- **Política de escalonamento**: *FCFS*, *Round Robin (RR)*, *Least Commands First (LCF)* e *Least Execution Time (LET)*.

- **Grau de paralelismo**: 1, 2, 4 e 8 execuções simultâneas.
- **Modo de histórico**: no modo **`cold`** o ficheiro `historical.bin` é eliminado antes de cada *run*, já no modo **`warm`** o histórico é pré-aquecido com uma *workload* invertida antes das repetições de medição.
- **Número de repetições por configuração**: 3, de forma a reduzir a variância introduzida por fatores externos ao sistema.

A escolha de utilizar comandos de espera (`sleep`) como carga de trabalho não foi arbitrária. O uso de tempos de execução simulados e previsíveis garante o determinismo dos testes. Esta abordagem isola o desempenho do algoritmo de fatores externos imprevisíveis (como atrasos de I/O no disco ou concorrência de memória, que ocorreriam ao usar comandos reais como ls ou grep). Ao garantir que a duração inerente de cada comando é matematicamente exata, asseguramos que as métricas de Slowdown e Turnaround medem única e exclusivamente o impacto das decisões de prioridade do próprio scheduler.

2.5.1.2. Carga de Trabalho

A carga de trabalho submetida em cada *run* é composta por quatro utilizadores com perfis distintos, representando desta forma um cenário heterogéneo e realista.

Utilizador	Nº de Comandos	Duração por Comando
1	8	100 ms
2	6	200 ms
3	4	400 ms
4	≈ 2 (1 a cada 4 rondas)	800 ms

Tabela 1: Distribuição de comandos por utilizador.

A submissão dos comandos é feita de forma paralela, com o intuito de simular uma situação real em que o *scheduler* tem de gerir múltiplos pedidos simultâneos. O Utilizador 4 possui a particularidade de ser mais esporádico, a fim de criar uma assimetria intencional que permite testar a equidade de cada política. É importante salientar que a configuração da *workload* tem impacto direto nos resultados obtidos, pelo que o estudo a seguir realizado tem como base a carga acima.

2.5.1.3. Histórico `Cold` e `Warm`

O modo **`cold`** garante que cada *run* começa sem qualquer contexto histórico, permitindo avaliar o desempenho puro de cada política sem influência de sessões anteriores.

Em contrapartida, no modo **`warm`**, antes de cada bloco de medições, é executada uma fase de pré-aquecimento com uma *workload* intencionalmente invertida (U4=20, U3=6, U2=4, U1=2 comandos), de forma a construir um `historical.bin` em que o Utilizador 4 é o mais pesado historicamente. Desta forma, quando as medições começam, os algoritmos que consultam o histórico (LCF e LET) partem de um estado com informação prévia que contraria o padrão da *workload* de medição, tornando o efeito do histórico persistente inequivocamente observável nos resultados.

2.5.1.4. Métricas de Avaliação

O *script* `analyze_stats.py` processa o ficheiro CSV gerado e calcula um conjunto de métricas por configuração. Para cada comando executado, o *controller* regista os instantes de submissão (`submit_time_ms`), de início de execução (`start_time_ms`) e de conclusão (`end_time_ms`), bem como a duração efetiva de execução (`duration_ms`). A partir destes valores são derivadas as seguintes métricas de análise:

Waiting Time: Intervalo entre a submissão de um comando e o início da sua execução (`waiting_ms = start_time - submit_time`). Representa o custo de espera na fila do *scheduler*.

Turnaround Time: Duração total entre a submissão e a conclusão de um comando (`turnaround_ms = end_time - submit_time`). Corresponde ao tempo total sentido pelo utilizador desde que submete o pedido até receber o resultado, isto é, a soma do tempo de espera, com o tempo de execução.

Slowdown (Normalized Turnaround): Quociente entre o *turnaround* e o tempo de execução. Um **slowdown** de 1,0 significa que o comando correu sem qualquer espera, por outro lado, um valor de 5,0 significa que o comando demorou cinco vezes mais do que o necessário. Esta métrica é especialmente relevante para comparar a penalização relativa entre comandos de durações muito distintas.

Throughput: Número de comandos executados por segundo (`|comandos| / (last_end - first_submit)`) por *run*. Mede a capacidade de processamento global do sistema e é independente da equidade entre utilizadores.

Coeficiente de Variação (CV): Quociente `desvio_padrão/média` aplicado às métricas de *waiting time*, *turnaround time* ou **slowdown**. Um CV baixo indica que os valores são homogéneos, isto é, todos os comandos esperam aproximadamente o mesmo tempo. Em contrapartida, um CV alto indica grande dispersão, podendo ser sinal de injustiça entre comandos ou utilizadores.

Índice de Jain: Métrica de equidade com intervalo [0, 1], definida como $(\Sigma x)^2 / (n \times \Sigma x^2)$. Neste caso, um valor próximo de 1,0 indica que todos os comandos receberam um tratamento semelhante, ao passo que um valor baixo indica concentração do recurso em poucos utilizadores. É complementar ao CV e permite uma leitura normalizada da equidade.

É ainda importante salientar que o *CV* e o *índice de Jain* são calculados sobre a população de todos os comandos e não por utilizador. Posto isto, devem ser interpretados em conjunto com os gráficos de equidade por utilizador, que revelam situações em que uma distribuição aparentemente homogénea ao nível agregado esconde disparidades significativas entre utilizadores específicos.

2.5.2. Análise de Resultados

Nota: Salvo indicação em contrário, os gráficos desta secção agregam os resultados dos modos *cold* e *warm* conjuntamente. A comparação explícita entre os dois modos é apresentada na secção de Histórico Persistente.

2.5.2.1. Impacto do Paralelismo

O fator com maior influência nos resultados é, inequivocamente, o grau de paralelismo:

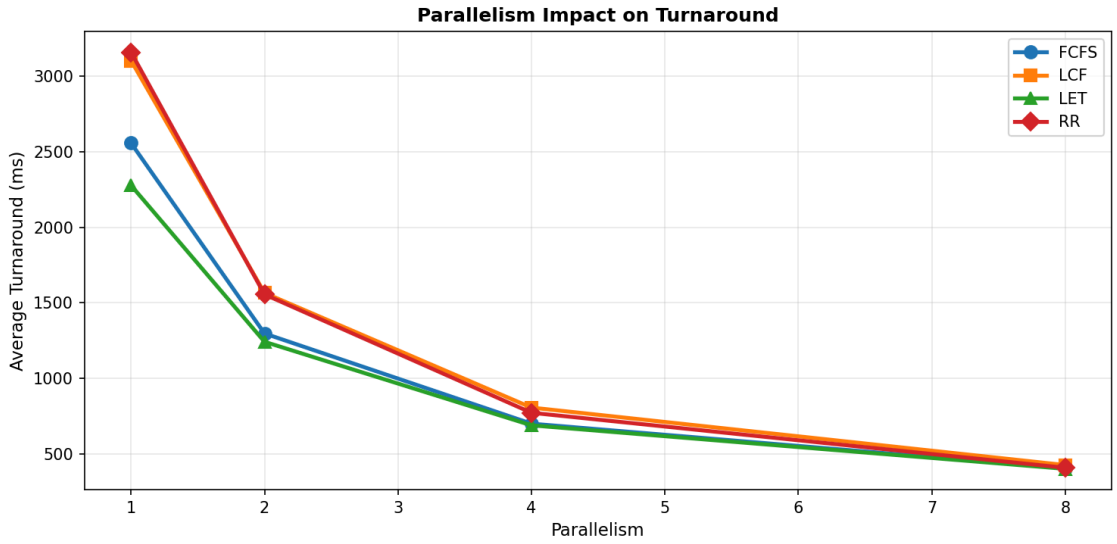


Figura 2: Impacto do paralelismo no *turnaround*

Como se verifica na figura, o aumento do número de execuções paralelas de 1 para 8, reduz o *turnaround time* em cerca de 83% para todas as políticas de escalonamento. Ora, isto permite concluir que o impacto do algoritmo é máximo quando o paralelismo é baixo, e perde relevância com o aumento de *slots* disponíveis para execução em simultâneo.

No que diz respeito ao *throughput*, o crescimento é aproximadamente linear com o paralelismo para todos os algoritmos, o que confirma que o sistema escala de forma eficiente.

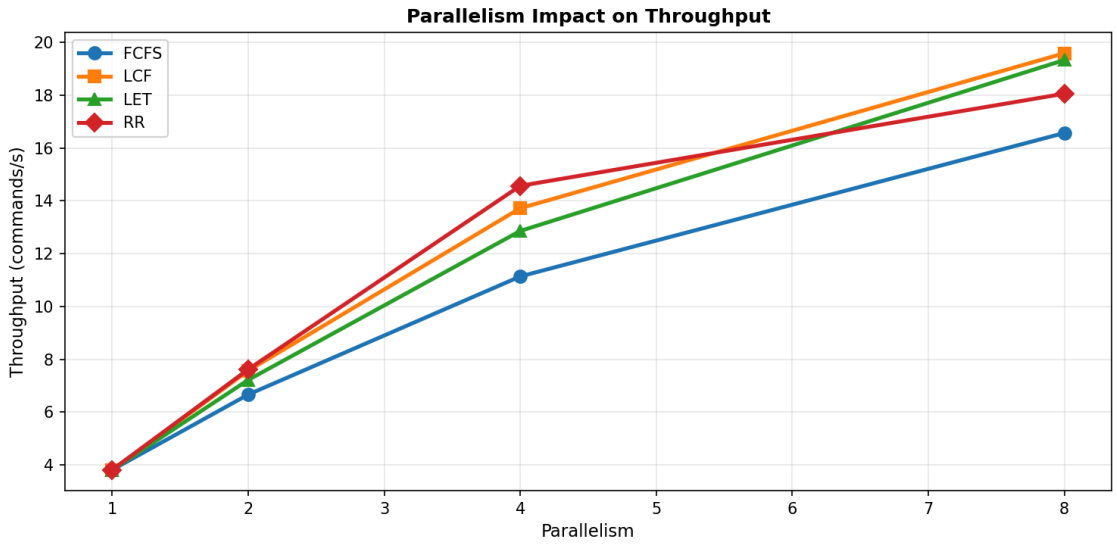


Figura 3: Impacto do paralelismo no *throughput*

Vale a pena salientar a diferença notável no *FCFS*, que apresenta valores de *throughput* significativamente menores a alto paralelismo. Este comportamento deve-se ao facto do algoritmo utilizar a ordem de chegada, o que acaba por misturar comandos curtos e longos nos mesmos *slots* de execução, e que leva a bloqueios na *queue*.

2.5.2.2. Comparação de Políticas a Baixo Paralelismo

Tal como visto anteriormente, a importância do algoritmo selecionado verifica-se principalmente com um nível de paralelismo 1.

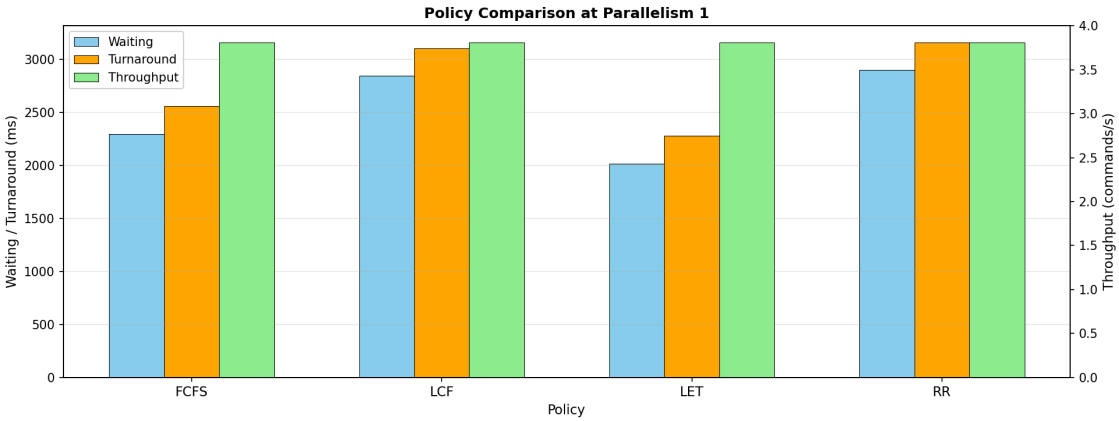


Figura 4: Desempenho por política a paralelismo 1

No gráfico acima podemos concluir que, o *LCF* e o *RR* apresentam os piores valores de *waiting* e *turnaround* (~2700ms e ~3090ms para o LCF; ~2800ms e ~3100ms para o RR), enquanto o *LET* obtém os melhores (~2050ms e 2250ms). O *FCFS* situa-se numa posição intermédia.

Este fraco desempenho do *LCF* neste regime pode ser explicado pela sua lógica, visto que este prioriza utilizadores com menos comandos históricos, atrasando os utilizadores mais ativos. Neste caso, o utilizador 1 (8 comandos rápidos) espera que os Utilizadores 3 e 4 (poucos comandos, mas longos) sejam servidos primeiro. Por outro lado, o *LET* considera o tempo de *CPU* efetivamente consumido, redistribuindo a prioridade de forma proporcional à carga real e evitando esse mesmo problema. Por fim, é de notar que o throughput se mantém constante (aproximadamente 3.8 comandos/s) independentemente da política de escalonamento escolhida. Uma vez que o grau de paralelismo é 1, o sistema processa os pedidos de forma estritamente sequencial (apenas um slot de execução). Como a carga de trabalho submetida é constante (20 comandos com a mesma soma total de tempos de sleep), a política de escalonamento altera apenas a ordem pela qual os comandos são despachados, mas não o tempo total necessário para concluir o lote inteiro. Consequentemente, a taxa de comandos concluídos por segundo permanece matematicamente idêntica para todos os algoritmos neste regime.

2.5.2.3. Penalização Relativa (*Slowdown*)

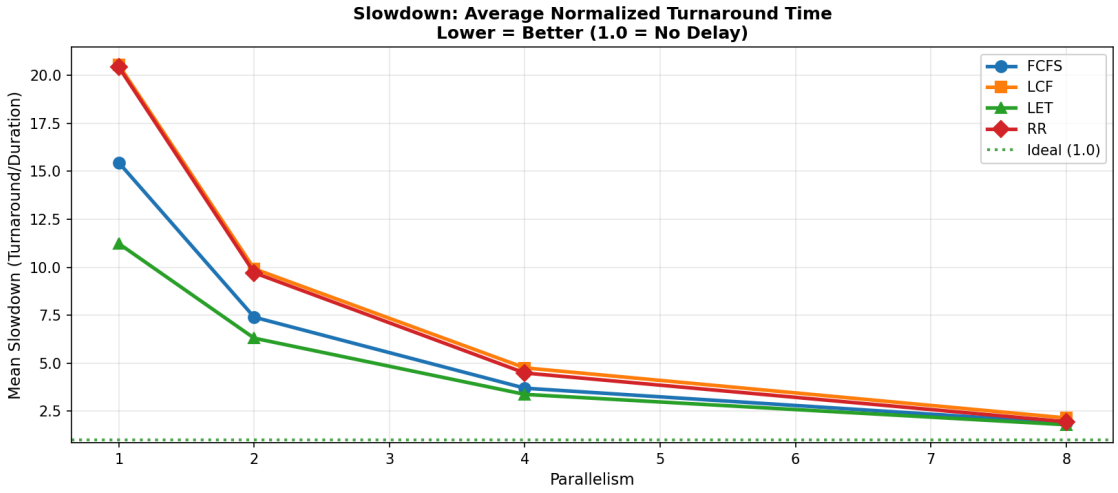


Figura 5: *Slowdown* por política para diferentes paralelismos.

O *Slowdown* médio (*turnaround/duration*) é a métrica que melhor representa a penalização relativa imposta pelo *scheduler*. Tal como se verifica no gráfico, com paralelismo 1, o *LCF* atinge um valor médio de aproximadamente 21x, o que significa que os comandos demoram 21 vezes mais o seu tempo inerente, maioritariamente em espera. Por sua vez, o algoritmo com melhores resultados é o *LET*, que atinge aproximadamente 11x. Ademais, verifica-se que todas as políticas convergem para ~2x quando P=8, aproximando-nos do cenário ideal onde o *slowdown* é 1.

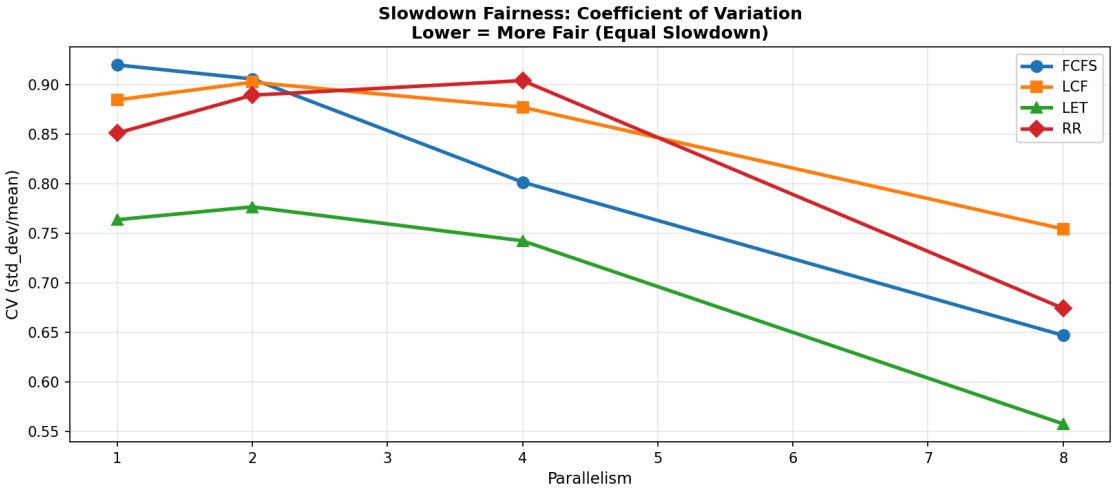


Figura 6: Coeficiente de variação do *Slowdown* por política para diferentes paralelismos.

Utilizando o coeficiente de variação do *slowdown*, podemos concluir qual é a política que distribui esta penalização de forma mais uniforme. Pelo gráfico, percebemos que o *LET* apresenta de forma consistente o *CV* mais baixo (~0,76 com P=1 e ~0.56 com P=8), demonstrando que todos os comandos sofrem uma penalização semelhante e independente da sua duração ou do utilizador. Além disso, verifica-se que o *LCF* mantém valores intermédios estáveis, ao contrário dos restantes algoritmos.

2.5.2.4. Justiça entre Utilizadores

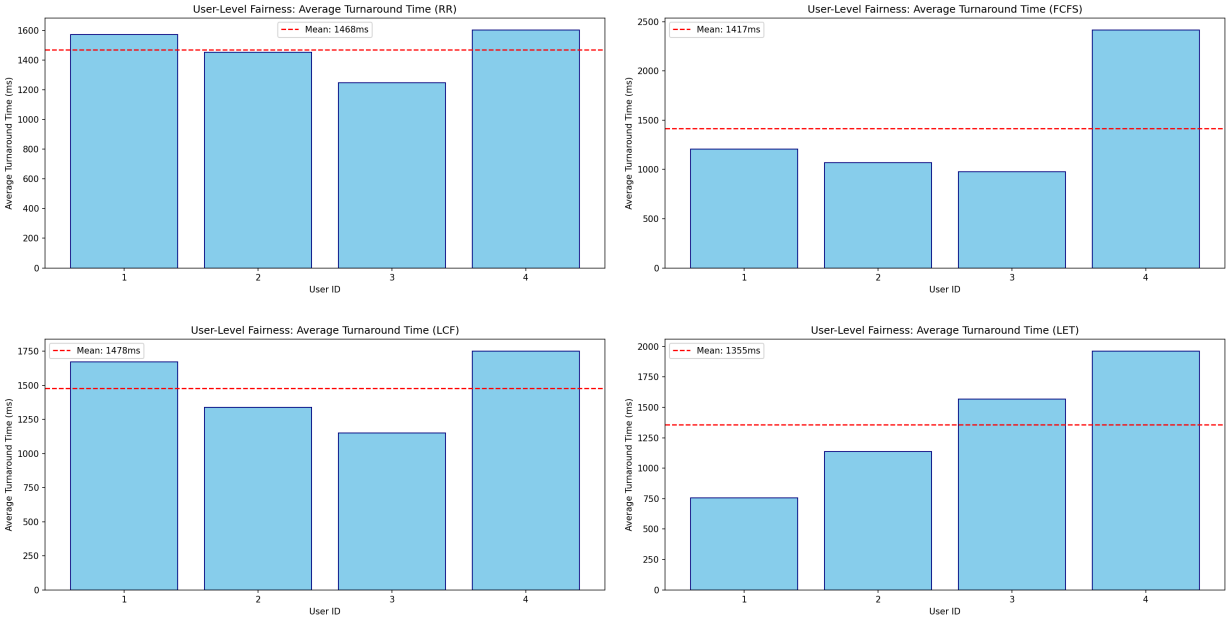


Figura 7: Justiça entre Utilizadores em cada política.

O *Round Robin* é a política mais justa ao nível do utilizador, onde os quatro utilizadores apresentam tempos de *turnaround* muito próximos (entre ~1250ms e ~1550ms), com um desvio reduzido em relação à média de ~1459ms. Isto deve-se ao facto do seu mecanismo de rotação circular garantir uma fatia de execução a cada utilizador, independentemente da quantidade ou duração dos seus pedidos.

Já o *FCFS* apresenta um comportamento aceitável para os utilizadores 1 a 3 (~1200ms-980ms), mas penaliza severamente o Utilizador 4 (~2400) uma vez que os seus pedidos chegam de forma esporádica, e têm a maior duração, acabando consequentemente com maior *waiting time*.

Em contrapartida, no caso do LCF, verifica-se uma dinâmica peculiar: tanto o Utilizador 1 (o mais ativo) como o Utilizador 4 (o mais esporádico) apresentam os maiores tempos de turnaround (cerca de 1650ms e 1750ms, respetivamente), enquanto o Utilizador 3 obtém o menor tempo (1150ms). O elevado turnaround do Utilizador 1 é diretamente justificado pela natureza do algoritmo, que penaliza utilizadores com muitos comandos históricos, forçando os seus pedidos rápidos a tempos de espera severos na fila. Por outro lado, embora o Utilizador 4 tenha prioridade máxima de escalonamento (por ter o menor número de comandos), o seu tempo de execução inerente é muito elevado (800ms). Como as suas submissões são esporádicas e o sistema não é preemptivo, este acaba por ter de esperar que os slots de execução fiquem livres, o que, somado à sua longa execução, inflaciona o turnaround. O Utilizador 3 acaba por beneficiar do ponto de equilíbrio ideal entre uma prioridade favorável na fila e um tempo de execução moderado.

Distingue-se ainda o *LET* onde o utilizador 1 tem menor *turnaround time* (~750ms), e os utilizadores 3 e 4 os maiores. Isto é justificado pelo facto do utilizador 1 submeter comandos de 100ms, pelo que mesmo com algum *waiting time*, o *turnaround time* é baixo. Ora esta política garante equidade em termos de tempo de *CPU* consumido, e não de *turnaround time*, pelo que utilizadores com pedidos mais longos, acumulam *turnaround times* maiores.

2.5.2.5. Métricas Agregadas de Justiça

Os gráficos de coeficiente de variação e do Índice de *Jain* sobre a população total de comandos revelam um resultado aparentemente contraditório, já que o *LCF* surge como a política "mais justa" por estas métricas, com o menor *CV* de *turnaround time* (~0,512 com P=1) e o maior Índice de *Jain* (~0,80 com P=1).

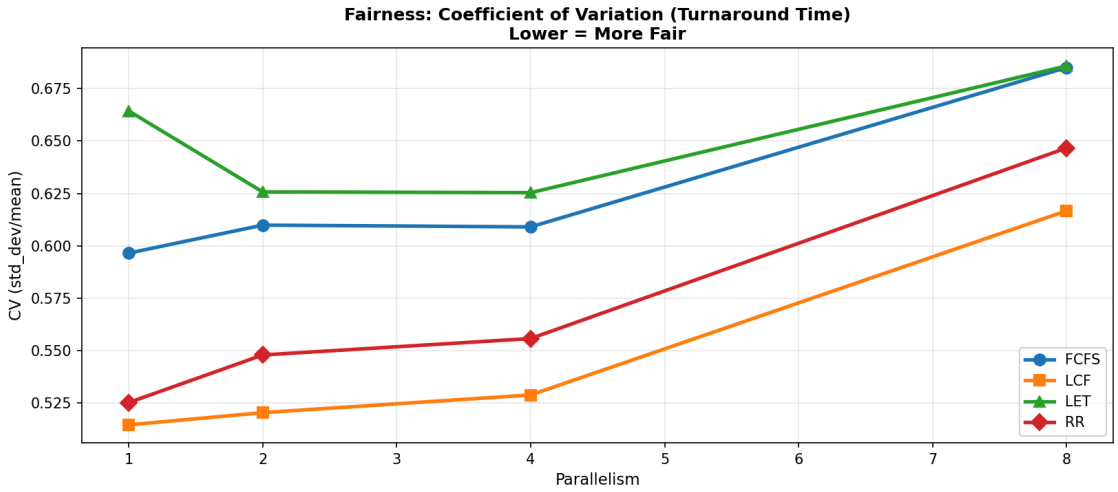


Figura 8: Coeficiente de Variação de *turnaround time* por paralelismo.

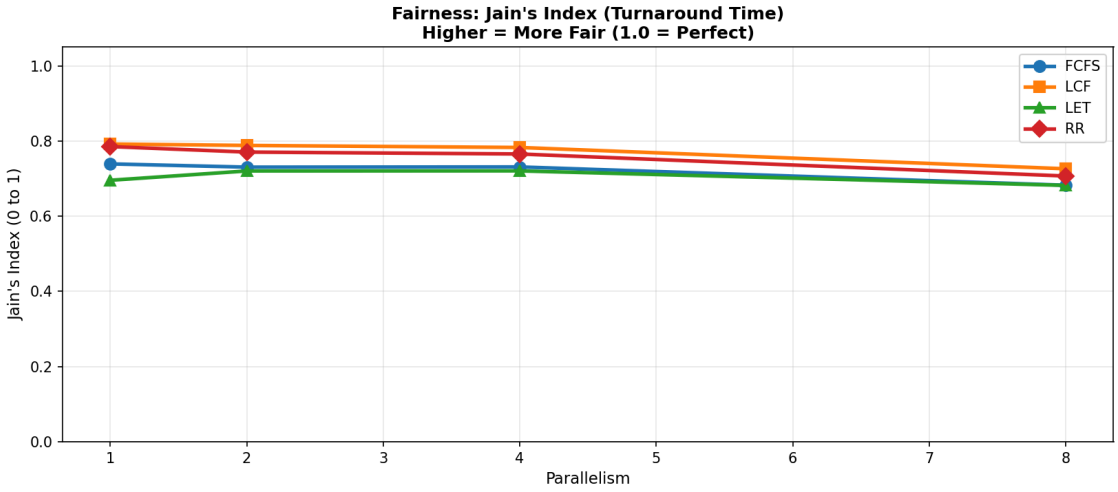


Figura 9: Índice de Jain por paralelismo.

Este resultado é a prova do determinismo desta política. Ao ordenar os utilizadores de forma estrita, o LCF cria grupos de comandos com turnaround times extremamente homogéneos dentro de cada utilizador. Uma vez que o Utilizador 1 e o Utilizador 2 representam 70% da população total de comandos (14 em 20), o facto de os seus comandos sofrerem tempos de retorno consistentes e idênticos entre si faz com que a variância estatística global desça drasticamente. Esta elevada consistência populacional aumenta o Índice de Jain ao nível do comando, mascarando a forte discrepância que existe ao nível dos utilizadores.

2.5.2.6. Impacto do Histórico Persistente

A comparação entre os modos *cold* e *warm* demonstra que o *LET* e o *LCF* beneficiam de forma significativa do histórico persistente, ao contrário do *FCFS* e do *RR* que, por não o consultarem, apresentam valores praticamente idênticos em ambos os modos.

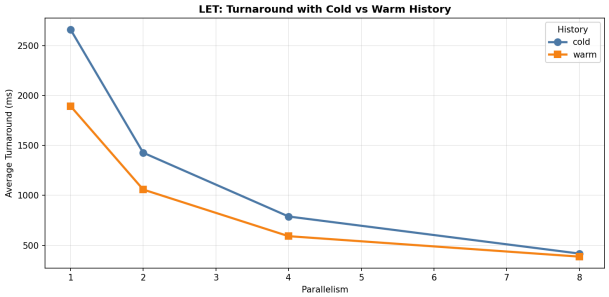


Figura 10: Impacto do histórico no *turnaround* do *LET*.

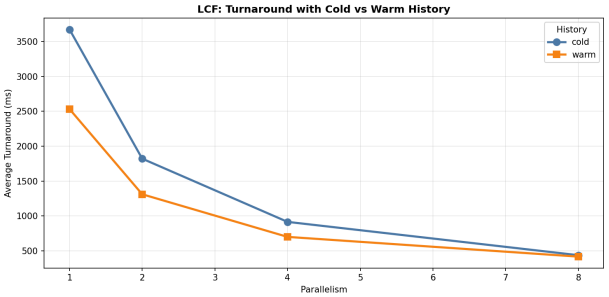


Figura 11: Impacto do histórico no *turnaround* do *LCF*.

Como se observa, o *LET* no modo *warm* reduz o *turnaround* em ~26% face ao modo *cold* (de ~2660ms para ~1900ms a P=1). Já o *LCF* no modo *warm* sofre uma redução ~28% (de ~3660ms para ~2520ms a P=1). Esta melhoria é uma consequência direta do pré-aquecimento do histórico com uma *workload* invertida, já que os algoritmos são iniciados com informação que lhes permite tomar decisões de prioridade imediatas, em vez de as inferir progressivamente durante a execução.

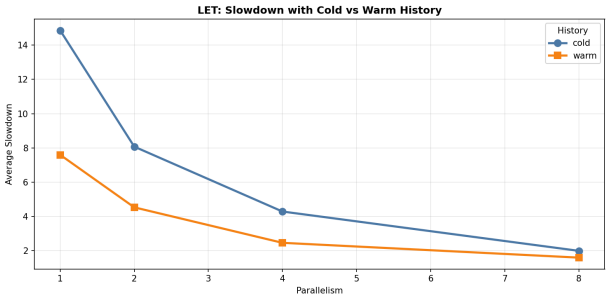


Figura 12: Impacto do histórico no *slowdown* do *LET*.

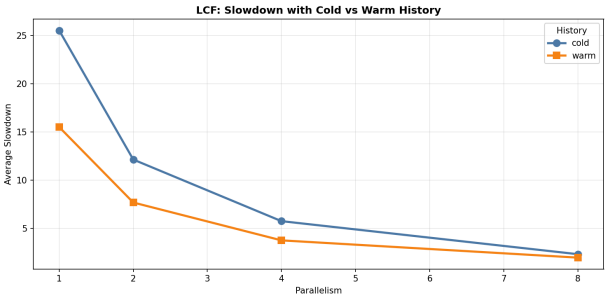


Figura 13: Impacto do histórico no *slowdown* do *LCF*.

Verifica-se ainda o mesmo padrão, de forma mais acentuada, no *slowdown*. No caso do *LET* em modo *warm*, desce de ~15× para ~7,5× a P=1 (~49%), e do *LCF*, em modo *warm* de ~25,5× para ~15,5× (~39%). A maior melhoria no *slowdown* do que no *turnaround* indica que os principais beneficiários são os utilizadores com comandos curtos, cujo *slowdown* é mais sensível a pequenas variações no tempo de espera.

É contudo importante contextualizar estes resultados: a melhoria observada no modo warm é amplificada pelo design do teste, cujo histórico de pré-aquecimento foi construído para penalizar o Utilizador 4, não refletindo necessariamente o benefício gradual que o histórico proporciona em produção ao longo de sessões repetidas. O inverso também se verifica: um histórico desatualizado pode degradar as decisões de escalonamento, penalizando indefinidamente um utilizador que abandonou o sistema mas acumulou peso histórico no passado.

2.5.3. Conclusão da Análise

Política	Waiting/Turnaround	Throughput (P=8)	Slowdown (P=1)	Justiça por Utilizador	Impacto do Histórico
LET	**Melhor**	2º	**Melhor**	Moderada	**Muito significativo**
FCFS	2º	**Pior**	2º	Moderada	Nenhum
RR	3º	3º	3º	**Melhor**	Nenhum
LCF	**Pior**	**Melhor**	**Pior**	**Pior**	**Significativo**

Tabela 2: Conclusões da análise por algoritmo.

Os valores recolhidos após a execução dos testes permite tirar algumas conclusões em relação a cada uma das políticas de escalonamento:

Primeiramente, verifica-se que o *LET* é a política com o melhor desempenho global, minimizando o *waiting* e *turnaround time*, bem como o *slowdown* (~11× a P=1, face a ~21× do *LCF* e ~20× do *RR*). Esta é também a política onde se verifica um maior impacto do histórico persistente (~45% de redução no *slowdown*), confirmando que a complexidade adicional (*O(n)* por decisão durante a implementação) é justificada.

De seguida, o *Round Robin* é a política mais justa ao nível do utilizador em termos de turnaround time, garantindo que nenhum utilizador é constantemente prejudicado, independentemente do seu padrão de submissão. Esta deve ser a escolha natural quando a equidade sentida é o critério dominante.

Por sua vez, o *FCFS* oferece simplicidade e previsibilidade, com a garantia de que nenhum comando é desprezado em relação a um que chegou depois. A sua limitação principal é a penalização de utilizadores esporádicos com comandos longos e o menor *throughput* a alto paralelismo (16,5 cmd/s face a 19,5 do LCF e 19,3 do *LET*).

Por fim, o *LCF* apresenta o pior desempenho a baixo paralelismo e penaliza os utilizadores mais ativos por não considerar a duração dos comandos. Beneficia significativamente do histórico persistente (37% de redução no *slowdown*), mas a sua lógica baseada na contagem de comandos permanece sensível à composição do histórico. Existe ainda um risco subjacente de *starvation* para utilizadores com alta frequência em sessões longas, caso o histórico acumule uma discrepância irreversível.

3. Conclusão

O desenvolvimento deste trabalho prático permitiu consolidar o conhecimento sobre os mecanismos de comunicação entre processos em sistemas *UNIX*, em particular o uso de *Named Pipes* como canal exclusivo de comunicação entre componentes independentes. A opção de estruturas de tamanho fixo para o protocolo de mensagens revelou-se uma decisão acertada, simplificando substancialmente a lógica de leitura e eliminando ambiguidades na fronteira entre mensagens.

Além disso, a arquitetura modular adotada, com o *dispatcher* independente das políticas de escalonamento através de uma interface comum, facilitou a implementação e o teste independente de cada algoritmo, e torna trivial a adição de novas políticas no futuro. A separação entre o *runner*, como processo efémero orientado a uma única tarefa, e o *controller*, como servidor de longa duração com estado persistente, reflete boas práticas de design de sistemas concorrentes.